

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_04.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Define and call functions in Python
- Use parameters and return values
- Understand variable scope (local vs global)
- Structure a program into reusable functions
- Organize functions in a Python module

Lab Plan

Phase	Content	Duration
Welcome	Review of functions	10 min
Exercise 1	Function calculate_average	15 min
Exercise 2	Custom max/min functions	15 min
Exercise 3	Palindrome function	20 min
Exercise 4	Exercise generator	20 min
Exercise 5	Statistical calculations module	10 min

Prerequisites

- Labs 01-03 completed (variables, conditions, loops)
- Concept of function covered in class (def, return)

Exercise 1 — Function calculate_average (15 min)

Problem statement: Create a script `moyenne_fonction.py` that defines a function `calculate_average(list)` that takes a list of numbers and returns their average. The main program asks the user for 5 grades, calls the function, and displays the result.

Expected output:

```
Grade 1 : 12
Grade 2 : 15
Grade 3 : 10
```

```
Grade 4 : 18
Grade 5 : 14
Average : 13.80/20
```

Solution:

```
def calculate_average(lst):
    """Returns the average of elements in a list."""
    if len(lst) == 0:
        return 0
    return sum(lst) / len(lst)

grades = []
for i in range(5):
    grade = float(input(f"Grade {i + 1} : "))
    grades.append(grade)

average = calculate_average(grades)
print(f"Average : {average:.2f}/20")
```

Exercise 2 — Custom max/min functions (15 min)

Problem statement: Create a script minmax.py that defines (without using max()/min()):

1. find_max(list) — returns the maximum value
2. find_min(list) — returns the minimum value
3. find_extremes(list) — returns (min, max) in a single function

The program asks for 4 numbers and displays min, max, and the range.

Solution:

```
def find_max(lst):
    max_val = lst[0]
    for val in lst:
        if val > max_val:
            max_val = val
    return max_val

def find_min(lst):
    min_val = lst[0]
    for val in lst:
        if val < min_val:
            min_val = val
    return min_val

def find_extremes(lst):
    return find_min(lst), find_max(lst)

numbers = []
for i in range(4):
    n = float(input(f"Number {i + 1} : "))
    numbers.append(n)

min_val, max_val = find_extremes(numbers)
range_val = max_val - min_val
print(f"Min : {min_val}, Max : {max_val}, Range : {range_val:.1f}")
```

Exercise 3 — Palindrome function (20 min)

Problem statement: Create a script `palindrome.py` that:

1. Defines `is_palindrome(word)` — returns True if the word reads the same forwards and backwards (case-insensitive)
2. Defines `test_sentence(sentence)` — tests each word in a sentence
3. The main program asks for a sentence and displays which words are palindromes

Solution:

```
def is_palindrome(word):
    word = word.lower()
    return word == word[::-1]

def test_sentence(sentence):
    for word in sentence.split():
        if is_palindrome(word):
            print(f'"{word}" → Yes ✓')
        else:
            print(f'"{word}" → No')

test_sentence(input("Sentence : "))
```

Function call flow (palindrome):

```
flowchart TD
    A[Start] --> B[Enter sentence]
    B --> C[Call test_sentence]
    C --> D[Split into words]
    D --> E[For each word]
    E --> F[Call is_palindrome]
    F --> G[Convert to lowercase]
    G --> H{word == reverse ?}
    H -->|Yes| I[Display ✓]
    H -->|No| J[Display No]
    I --> K[End loop]
    J --> K
    K --> L[End]
```

Exercise 4 — Random exercise generator (20 min)

Problem statement: Create a script `generateur.py` that generates mental arithmetic exercises for students. The program:

1. Defines `generate_question()` which returns (a, op, b, answer) randomly
2. Defines `ask_question(question)` which asks the question and returns True if correct
3. Offers 5 questions with addition, subtraction, multiplication
4. Displays the score out of 20 at the end

Expected output:

```
=== Mental arithmetic generator ===
12 + 7 = 19 ✓ Correct!
15 - 8 = 7 ✓ Correct!
6 × 9 = 54 ✓ Correct!
...
Score : 16/20
```

Solution:

```

import random

def generate_question():
    a = random.randint(1, 20)
    b = random.randint(1, 20)
    op = random.choice(["+", "-", "*"])
    if op == "+":
        return a, op, b, a + b
    elif op == "-":
        a, b = max(a, b), min(a, b)
        return a, op, b, a - b
    else:
        a, b = random.randint(2, 9), random.randint(2, 9)
        return a, op, b, a * b

def ask_question(a, op, b):
    try:
        return int(input(f"{a} {op} {b} = "))
    except ValueError:
        print("Invalid answer.")
        return None

score = 0
for _ in range(5):
    a, op, b, correct = generate_question()
    ans = ask_question(a, op, b)
    if ans == correct:
        print("✓ Correct!")
        score += 1
    else:
        print(f"✗ Wrong. Answer: {correct}")

print(f"\nScore : {score * 4}/20")

```

Exercise 5 — Statistical calculations module (10 min)

Problem statement: Create a file `stats_utils.py` containing a module with the functions:

1. `mean(list)` — calculates the mean
2. `median(list)` — calculates the median
3. `std_dev(list)` — calculates the standard deviation
4. `stats_summary(list)` — returns all stats at once

Then create a script `test_stats.py` that imports the module and tests it with a list of grades.

Solution — `stats_utils.py`:

```

def mean(lst):
    if not lst:
        return 0
    return sum(lst) / len(lst)

def median(lst):
    sorted_lst = sorted(lst)
    n = len(sorted_lst)
    if n % 2 == 0:
        return (sorted_lst[n // 2 - 1] + sorted_lst[n // 2]) / 2
    return sorted_lst[n // 2]

def std_dev(lst):

```

```

if not lst:
    return 0
m = mean(lst)
variance = sum((x - m) ** 2 for x in lst) / len(lst)
return variance ** 0.5

def stats_summary(lst):
    return {"min": min(lst), "max": max(lst),
            "mean": mean(lst), "median": median(lst),
            "std_dev": std_dev(lst), "count": len(lst)}

```

Solution — test_stats.py:

```

import stats_utils

grades = [12, 15, 10, 18, 14, 16, 11, 13]
summary = stats_utils.stats_summary(grades)

for key, value in summary.items():
    if isinstance(value, float):
        print(f" {key} : {value:.2f}")
    else:
        print(f" {key} : {value}")

```

Module diagram (statistical calculations):

```

flowchart LR
    A["stats_utils.py"] --> B[mean]
    A --> C[median]
    A --> D[std_dev]
    A --> E[stats_summary]
    B --> F[test_stats.py]
    C --> F
    D --> F
    E --> F
    F --> G[Results]

```

Self-assessment quiz

- What keyword is used to define a function in Python?
 - a) function
 - b) def ✓
 - c) define
- What does a function without an explicit return return?
 - a) 0
 - b) None ✓
 - c) False
- How do you call a function named calculate?
 - a) call calculate()
 - b) calculate() ✓
 - c) run calculate()
- What is the scope of a variable defined inside a function?

- a) Global (everywhere in the program)
- b) Local (inside the function only) ✓
- c) Dynamic

5. A .py file containing importable functions is called:

- a) A library
- b) A module ✓
- c) A script

Grading Rubric Lab 04

Criterion	Weight	Not acquired (0)	Partial (0.5)
Function definition (<code>def</code>)	20%	Does not define functions	Incorrect syntax
Parameters and <code>return</code>	20%	Does not use parameters	Partial use
Calls and reuse	20%	Does not call functions	Erroneous calls
Docstrings and documentation	15%	No docstrings	Partial docstrings
Module and import	15%	No module	Incorrect imports
Code structuring	10%	Unstructured code	Partial structuring

Session schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome and functions review	10 min	10 min
Exercise 1 — calculate_average	15 min	25 min
Exercise 2 — Custom max/min	15 min	40 min
Exercise 3 — Palindrome	20 min	60 min
Exercise 4 — Exercise generator	20 min	80 min
Exercise 5 — Statistics module	10 min	90 min

Pedagogical Note ENS

Teaching transposition

This concluding lab on functions and modularity shows how abstraction (encapsulating code) allows building reusable programs. The exercise generator (ex. 4) illustrates how Python creates interactive teaching materials for primary and middle school. The statistics module (ex. 5) prepares students to create their own grade analysis tools. The instructor can emphasize the importance of docstrings and invite students to adapt these exercises for different school levels (cycle 3, cycle 4, high school).

Trainer advice

Ensure that students understand the difference between parameter and argument. Use the analogy: a function is like a cooking recipe (parameters = ingredients, body = steps, return = finished dish). For NSI students, the statistics module can be enriched with median, standard deviation, and quartiles.

Extension

Create an "Exercise sheet generator" module that randomly produces mathematics exercises (middle school level) with solutions, exportable to PDF via the `reportlab` library.